

ANNEXURE D: QA TEST SUITE & MIGRATION SUMMARY REPORT

System Name: Educational Institution Gate Monitoring & Access Control System
Document Code: QA-MIG-001
Version: 1.0.0
Date: August 17, 2026
Classification: Confidential - Quality Assurance & Audit Summary

1. Database Migration Execution Summary

The database migration roadmap transitions the Gate Monitoring System from initial prototype state to an enterprise-grade, security-hardened Operational POC.

Migration File	Migration Name	Applied Features & Schema Modifications	Security Impact
0001_initial_schema.sql	Initial Schema Base	Primary tables (<code>gates</code> , <code>users</code> , <code>students</code> , <code>gate_passes</code> , <code>gate_logs</code> , <code>campus_occupancy</code> , <code>alerts</code> , <code>audit_logs</code> , <code>notifications</code> , <code>sessions</code>) and B-Tree indexes.	Established foundational database relationships and constraints.
0002_functions_triggers_rls.sql	Logic & RLS Base	Basic PL/pgSQL helper functions and preliminary RLS policy setup.	Initiated Row Level Security policy structure on core tables.
0003_complete_schema_fixes.sql	Schema Alignment	Adjusted enum constraints, fixed nullability columns, added <code>login_identifier</code> and password hashing columns.	Resolved data integrity mismatches between Next.js models and SQL schema.
0004_supabase_auth_integration.sql	Auth Layer Integration	Synchronized local <code>users</code> table with Supabase <code>auth.users</code> provider.	Enabled dual-token validation and secure password management.
0005_security_hardening.sql	Security Hardening	Hardened <code>SECURITY DEFINER</code> functions, added missing supervisor/operator RLS policies, and added <code>trigger_notify_session_invalidation</code> .	Prevented privilege escalation, enforced strict gate data isolation, and enabled live session invalidation.

2. Security Verification & Audit Findings Remediation

During formal security auditing, multiple vulnerability vectors were identified and remediated across the codebase.

+-----+ SECURITY VULNERABILITY REMEDIATION +-----+	
[CRITICAL-001]	Missing Account Status Check ==> FIXED (Middleware + Auth)

2.1 Fixed Critical Vulnerabilities

3. QA Test Suite & Automated Verification Framework

3.1 Suite Inventory & Coverage

3.2 Key Test Scenarios & Results

- **Objective:** Verify that locked or suspended accounts are instantly barred from API endpoints.
- **Execution:** User account status transitioned from **ACTIVE** to **SUSPENDED**. API request dispatched with existing token.
- **Result:** **403 Forbidden** returned with error payload **ACCOUNT_SUSPENDED**. Session marked **active = FALSE**. **PASS.**

Test Case 2: Operator Identity Forgery Defense (TEST_authorization_security.ts)

- **Objective:** Verify that a malicious client cannot inject a false `operatorId` in scan logs.
- **Execution:** Operator sends `POST /api/gate/scan` with body payload `{ operatorId: "attacker-fake-uuid" }`.
- **Result:** Server ignores body parameter, binds scan log to operator ID matching JWT token. **PASS.**

Test Case 3: Pass Approval Workflow Isolation (TEST_authorization_security.ts)

- **Objective:** Verify students cannot self-approve gate passes.
- **Execution:** Authenticated student account dispatches `PUT /api/passes/[id]/approve`.
- **Result:** `403 Forbidden` returned. Permission `passes:approve` missing. **PASS.**

4. Operational QA Sign-Off & Verification Summary

- **Total Database Migrations:** 5 Applied Successfully (0 Pending).
- **Core Security Findings:** 100% Remediation Rate (0 Critical/High Remaining).
- **Test Suite Results:** 4/4 Test Suites Passed (100% Execution Success).
- **Compliance Status:** Ready for Operational Deployment & Field Testing.